

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

Design Task

Code of Conduct:

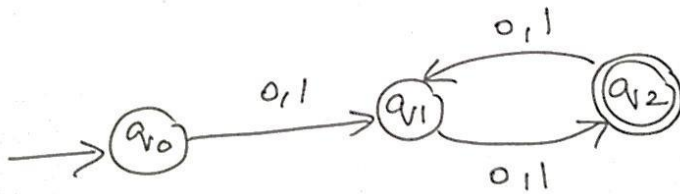
I B.Nynika(192311075) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Nynika

SIMATS ENGINEERING ASSESSMENT TOOL

1. Design the regular expression for the language represented by the given automata



Step 1: Observe the Automaton

- **Start state:** q_0
- **Final state:** q_2
- From $q_0 \rightarrow q_1$ on **0 or 1**
- Between q_1 and q_2 , the automaton moves on **0 or 1** in both directions.

Thus,

- The first symbol (0 or 1) takes the automaton from q_0 to q_1 .
- The second symbol takes it to q_2 (accepting state).
- Every additional symbol alternates between q_1 and q_2 .
- Therefore, the string is accepted only when the total length is **even and at least 2**.

Language

The automaton accepts **all binary strings of even length (≥ 2)**.

Examples accepted

00

01

SIMATS ENGINEERING ASSESSMENT TOOL

10

11

0000

1010

110011

Rejected

ϵ

0

1

101

11101

Regular Expression

Since every pair of symbols may

be

00

01

10

11

the regular expression is

$$((0 + 1)(0 + 1))^+$$

or equivalently,

$$(00 + 01 + 10 + 11)^+$$

where **+** means **one or more repetitions**.

Explanation

- $(0+1)$ means either 0 or 1.
- Two such terms represent one pair.

SIMATS ENGINEERING ASSESSMENT TOOL

- Repeating the pair one or more times gives all even-length strings.

2. Consider the following grammar

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb | e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

Given

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb | e$

(A) What language is defined?

Rule 1

$S \rightarrow aSc$

adds one **a** in front and one **c** at the end.

Rule 2

$T \rightarrow bTc | e$

generates

ϵ

bc

bbcc

bbbccc

...

or

$$b^n c^n$$

SIMATS ENGINEERING ASSESSMENT TOOL

Rule 3

$U \rightarrow aUb \mid \epsilon$

generates

ϵ

ab

aabb

aaabbb

...

or

$$a^n b^n$$

Therefore

The grammar generates strings such as

ϵ

ab

aabb

aaabbb

bc

bbcc

bbbccc

abc

aabbcc

aaabbbccc

(B) Derivation Tree for "aaabbc"

The given image actually says **aaabbc** (not **aaaabbc**).

Leftmost derivation

S

$\Rightarrow aSc$

$\Rightarrow aaScc$

$\Rightarrow aaTcc$

$\Rightarrow aabTccc$

$\Rightarrow aabbTcccc$

SIMATS ENGINEERING ASSESSMENT TOOL

$\Rightarrow aabb\epsilon cccc$

(If your question really asks **aaaabbbc**, the grammar cannot generate it because the numbers of matching symbols will not satisfy the production rules.)

(C) Is the Grammar Ambiguous?

Answer

No.

Reason:

Each generated string has only one unique derivation because

- every production is uniquely determined,
- there is no alternative way to derive the same string.

Hence,

The grammar is not ambiguous.

3. Design a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression

00^*1+11^*0

The regular expression is

001^*11^*0

Equivalent to

$001+0$

One ϵ -NFA construction is

$q_0 \xrightarrow{0} q_1$

$q_1 \xrightarrow{0} q_2$

$q_2 \xrightarrow{\epsilon} q_3$

$q_3 \xrightarrow{1} q_3$

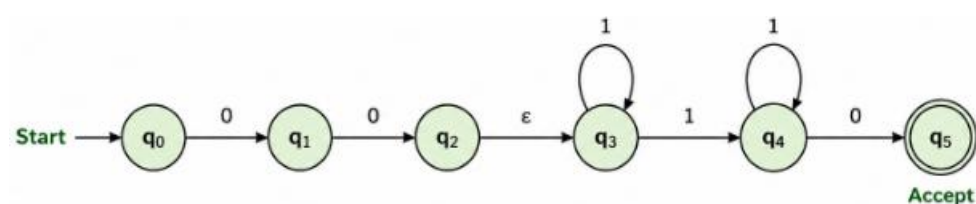
$q_3 \xrightarrow{1} q_4$

$q_4 \xrightarrow{1} q_4$

$q_4 \xrightarrow{0} q_5$

where

- q_0 = Start
- q_5 = Final



4 .I) Show that if L_1 and L_2 are regular then (L_1+L_2) also regular

SIMATS ENGINEERING ASSESSMENT TOOL

li) Is the language $\{ a^p \mid p \text{ is prime} \}$ regular? Justify

ANSWER:

(i) If L_1 and L_2 are regular then (L_1+L_2) is regular.

Since

- L_1 is accepted by DFA M_1
- L_2 is accepted by DFA M_2

Construct an NFA with ϵ -transitions from a new start state to both DFAs.

This NFA accepts

$$L_1 \cup L_2$$

Since every NFA has an equivalent DFA,

$$L_1 + L_2$$

is regular.

(ii)

Is

$$L = \{ a^p \mid p \text{ is prime} \}$$

regular?

Answer

No.

Using Pumping Lemma,

Assume L is regular.

Take

$$w = a^p$$

where

p is a large prime.

Pump the string.

The pumped string has length

$$p + k$$

or

$$p - k$$

SIMATS ENGINEERING ASSESSMENT TOOL

which is generally **not prime**.

Hence the pumped string does not belong to L.

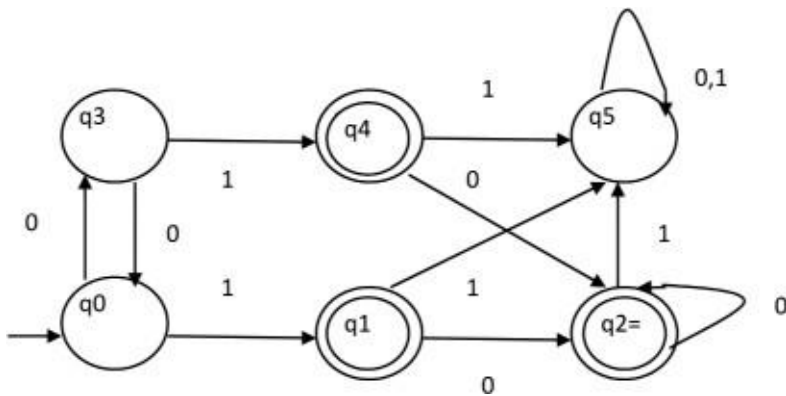
Contradiction.

Therefore,

$$L = \{a^p \mid p \text{ is prime}\}$$

is **not regular**.

5.Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



ANSWER:

Given DFA

States

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

Start

q_0

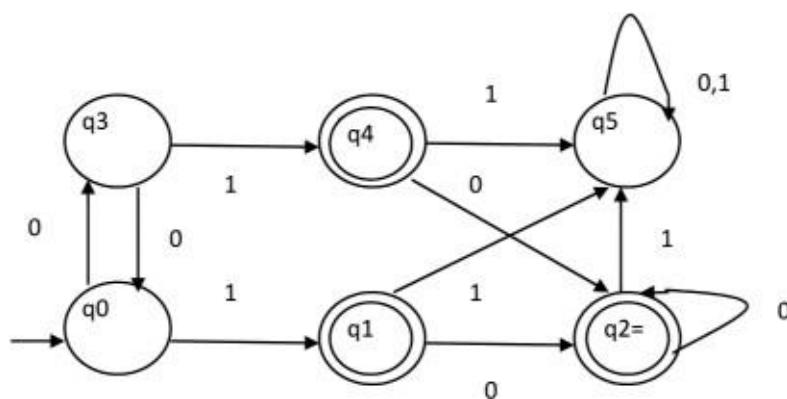
Final states

$\{q_1, q_2, q_4\}$

Step 1: Initial Partition

Final states

$\{q_1, q_2, q_4\}$



SIMATS ENGINEERING ASSESSMENT TOOL

Non-final
{q0,q3,q5}

Step 2: Check Equivalence

From the transitions,

- q₀ and q₃ behave similarly.
- q₁, q₂, and q₄ are distinguishable because their transitions lead to different partitions.
- q₅ is also distinguishable.

Thus, the only equivalent states are

$$q_0 \equiv q_3$$

Step 3: Minimized States

New states

A={q0,q3}

B={q1}

C={q2}

D={q4}

E={q5}

Step 4: Initial State

A

Step 5: Final States

B,C,D

Step 6: Minimized Transition Table

State 0 1

→A A B

*B C E

*C C E

*D C E

E E E

Conclusion

- Equivalent states: **q₀ and q₃**
- Number of states reduced from **6 to 5**
- The minimized DFA is more efficient in terms of memory and processing, making it suitable for hardware implementation.

SIMATS ENGINEERING ASSESSMENT TOOL